

Module 10 — REST API Design

High priority. “Design a REST API for X”, idempotency, and correct status codes come up in almost every backend interview. You already know HTTP from Express — this formalizes it.

Node.js bridge: Same HTTP semantics you used in Express; the difference is Java interviewers expect precise vocabulary (idempotency, HATEOAS awareness, versioning strategy).

10.1 REST Principles

REST = **RE**presentational **St**ate **T**ransfer. Constraints worth naming: - **Client-server + stateless** — no server session; each request carries all context (auth token, params). Enables horizontal scaling. - **Uniform interface** — resources identified by URIs (`/users/42`), manipulated via representations (JSON), standard verbs. - **Cacheable** — responses declare cacheability (`Cache-Control`, `ETag`). - **Layered system** — client can't tell if it talks to the server or a proxy/gateway. - **Resource-oriented naming**: nouns not verbs, plural collections — `GET /users/42/orders`, **not** `getUserOrders`.

Trap: “REST is just HTTP + JSON.” Interviewers want *statelessness*, *resource nouns*, and *correct verb/status usage*.

10.2 HTTP Methods, Idempotency & Safety (the key table)

Method	Purpose	Safe (no change)	Idempotent	Body
GET	read	Yes	Yes	no
POST	create / non-idempotent action	No	No	yes
PUT	full replace/create-at-id	No	Yes	yes
PATCH	partial update	No	usually No*	yes
DELETE	remove	No	Yes	maybe

* PATCH *can* be idempotent depending on the patch semantics.

Idempotency (must-explain concept)

An operation is **idempotent** if making it **N times has the same effect as once**. - `PUT /users/42 {full body}` — repeat → same final state → idempotent. - `POST /users` — repeat → creates duplicates → **not** idempotent. - **DELETE** — deleting twice: resource still gone (often 404 the second time, but state is idempotent).

Production pattern — idempotency keys: For `POST /payments`, clients send an **Idempotency-Key** header; the server stores the key + result so a retried request returns the original response instead of double-charging. Classic Stripe interview point.

Best Answer

“GET is safe and idempotent; PUT and DELETE are idempotent but not safe; POST is neither, so retries can duplicate. For non-idempotent POSTs like payments I use an idempotency key the server dedupes on, so client retries after a timeout don't double-charge.”

10.3 HTTP Status Codes (know the exact ones)

Range	Meaning	Common codes
2xx success		200 OK, 201 Created (+ Location), 202 Accepted (async), 204 No Content (delete)
3xx redirect		301 Moved Permanently, 304 Not Modified (caching/ETag)
4xx client error		400 Bad Request, 401 Unauthorized (authN), 403 Forbidden (authZ), 404 Not Found, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx server error		500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Traps: 401 vs 403 (not authenticated vs authenticated-but-forbidden); return **201 + Location** on create, **204** on delete with no body; **409** for conflicts (duplicate), **422** for semantic validation failure, **429** for rate limiting.

10.4 Pagination, Filtering, Sorting

- **Offset pagination:** `GET /users?page=2&size=20&sort=name,asc`. Simple; slow/inconsistent for deep pages on large tables. In Spring Data: `Pageable` → `Page<T>` (includes total count).
- **Cursor/keyset pagination:** `?after=<lastId>&size=20`. Stable and fast for large datasets (used by Stripe/Slack). Preferred at scale.
- **Filtering:** query params `?status=ACTIVE&minAge=18` (or a query DSL / Specifications).
- **Sorting:** `?sort=field,dir`.
- Return metadata: total count / next-cursor, and consider envelope vs headers (`Link` header).

```
@GetMapping("/users")
Page<UserDto> list(@RequestParam(defaultValue="0") int page,
                  @RequestParam(defaultValue="20") int size,
                  @RequestParam(required=false) Status status,
                  @SortDefault(sort="createdAt", direction=Sort.Direction.DESC) Pageable pageable) {
    return service.search(status, pageable);
}
```

10.5 API Versioning

Strategy	Example	Notes
URI path (most common)	<code>/api/v1/users</code>	simplest, visible, cache-friendly
Header	<code>Accept: application/vnd.app.v2+json</code>	clean URLs, harder to test
Query param	<code>/users?version=2</code>	easy but muddies caching

Best answer: “URI versioning (`/v1`) is the pragmatic default — explicit and easy to route at the gateway. I version only on breaking changes, keep backward compatibility via additive changes, and deprecate old versions with a sunset policy.”

10.6 Good REST API design (senior checklist)

- Nouns + plurals, nested for relationships: `/customers/1/orders`.
- Consistent error envelope: `{ "code", "message", "traceId", "fieldErrors" }`.

- Validate input (`@Valid`) → 400/422; centralize with `@RestControllerAdvice`.
- Use DTOs, never expose JPA entities directly (avoids over-exposing fields + lazy issues).
- `ResponseEntity` for explicit status/headers; `201 + Location` on create.
- Document with OpenAPI/Swagger.
- Rate limit (429), paginate everything that can grow, support ETags for caching.

Module 10 — Top 25 Interview Questions (senior answers)

1. **What is REST?** Stateless, resource-oriented architecture over HTTP with a uniform interface.
2. **Is REST stateless? Why does it matter?** Yes — each request self-contained → horizontal scale.
3. **PUT vs POST?** Idempotent replace/create-at-id vs non-idempotent create.
4. **PUT vs PATCH?** Full replace vs partial update.
5. **What is idempotency?** N calls = 1 call's effect; GET/PUT/DELETE yes, POST no.
6. **How to make POST safe to retry?** Idempotency key dedup on the server.
7. **Which methods are safe?** GET (and HEAD/OPTIONS) — no state change.
8. **201 vs 200 vs 204?** Created (+Location) / OK / No Content.
9. **401 vs 403?** Unauthenticated vs forbidden.
10. **409 vs 422?** Conflict (duplicate/state) vs semantic validation failure.
11. **429?** Too Many Requests (rate limiting).
12. **502 vs 503 vs 504?** Bad gateway / unavailable / gateway timeout.
13. **Offset vs cursor pagination?** Simple vs stable-and-fast at scale.
14. **How to paginate in Spring Data?** `Pageable` → `Page/Slice`.
15. **How do you version APIs?** URI path (default), header, or query.
16. **When to bump the version?** Only breaking changes; keep additive changes compatible.
17. **Why DTOs over entities?** Decoupling, security, avoid lazy/serialization issues.
18. **How to design error responses?** Consistent envelope with code/message/traceId.
19. **What is HATEOAS?** Hypermedia links in responses (aware-of; rarely fully implemented).
20. **Idempotent DELETE returning 404 twice — ok?** Yes; final state is idempotent.
21. **ETag / 304?** Conditional caching to save bandwidth.
22. **Path vs query params?** Identify resource vs filter/paginate/sort.
23. **Bulk operations design?** Batch endpoint + partial success (207-style) semantics.
24. **How to secure a REST API?** AuthN (JWT/OAuth), AuthZ, HTTPS, rate limit, input validation.
25. **REST vs gRPC vs GraphQL?** Simple/ubiquitous vs high-perf RPC vs flexible client-driven queries.

Module 10 — Top Coding Questions

- Design + implement CRUD for `/api/v1/orders` with correct verbs/status codes.
- Add pagination, filtering, and sorting to a list endpoint.
- Implement idempotent payment creation with an `Idempotency-Key`.
- Build a consistent error response with `@RestControllerAdvice`.

Module 10 — Common Follow-ups

- “A client’s payment request times out and it retries — how do you prevent a double charge?”
- “You added a required field to the response — is that a breaking change? Version?”
- “Why not return the JPA entity directly?”

Module 10 — One-Page Cheat Sheet

```
REST: stateless, resource nouns (plural), uniform interface, cacheable
GET safe+idempotent | PUT/DELETE idempotent | POST neither | PATCH partial
Idempotency: N=1 effect. POST payments -> Idempotency-Key dedup (no double charge)
Status: 200 OK | 201 Created(+Location) | 204 No Content | 400 bad | 401 authN | 403 authZ |
      404 | 409 conflict | 422 validation | 429 rate limit | 500 | 502/503/504 gateway
Pagination: offset(page/size, Page) vs cursor(after=lastId, scalable)
Versioning: /api/v1 (default) | Accept header | query. Bump only on breaking change.
Use DTOs not entities; @Valid -> 400; @RestControllerAdvice error envelope; ResponseEntity
```

Module 10 — Mock Interview (answer, then continue)

1. "Design the REST API for an e-commerce order service — resources, verbs, status codes, pagination."
2. "Explain idempotency and how you'd make **POST /payments** safe under client retries."
3. "Give me the right status code for: create success, validation failure, duplicate, unauthorized, rate-limited."
4. "Offset vs cursor pagination — which for a 100M-row table and why?"
5. "How do you version an API and decide when a change is breaking?"

Continue to Module 11 when ready.